



Big Z

Surfando em Habitos Sustentaveis

João Pedro - 2417151
Luiz Carlos - 2410410
Ricardo André - 2417200



BigZ - Surfando em Habitros Sustentaveis

T300 - Programacao Funcional Universidade de Fortaleza - UNIFOR Prof. Bruno Lopes

Equipe e Divisao de Modulos

Membro	Modulo	Requisitos Funcionais
Joao Pedro Mendes Moreira	Modulo A - Autenticacao e Perfil	RF01, RF02, RF03
Luiz Carlos	Modulo B - Gestao de Habitros	RF04, RF05, RF06
Ricardo Andre	Modulo C - Registro e Acompanhamento	RF07, RF08, RF09

Link do Video

Gravacao disponivel na pasta do Google Drive da equipe:

<https://drive.google.com/drive/folders/11NN6f6ez6BxT9AYyxfvGfxJBL2g-V7Mh?usp=sharing>

Repositorio

Codigo-fonte completo no GitHub:

<https://github.com/JPMendesM/BigZ>

Tecnologias Utilizadas

Tecnologia	Finalidade
Elixir 1.19 + Phoenix 1.8	Framework principal da aplicacao
Phoenix LiveView 1.1	Interface reativa sem JavaScript manual
Ecto + PostgreSQL (Supabase)	Persistencia e consultas relacionais
HEEx	Templating dos componentes e layouts
Tailwind CSS v4 + DaisyUI 5	Estilizacao e sistema de design
Phoenix.PubSub	Atualizacoes em tempo real (RF09 / RT05)
mix phx.gen.auth	Geracao do sistema de autenticacao (RT03)

Requisitos Implementados

Requisitos Tecnicos Obrigatorios (RT01-RT07)

Codigo	Requisito	Status	Arquivo principal
RT01	Phoenix com LiveView	Concluido	lib/bigz_web/router.ex
RT02	Banco de dados via Ecto (PostgreSQL)	Concluido	lib/bigz/habits.ex , lib/bigz/accounts.ex
RT03	Autenticacao via mix phx.gen.auth Layout com HEEEx +	Concluido	lib/bigz_web/user_auth.ex
RT04	Tailwind CSS LiveView com	Concluido	lib/bigz_web/components/layouts.ex
RT05	atualizacao em tempo real via PubSub Changesets com	Concluido	lib/bigz_web/live/community_live/index.ex
RT06	validacoes no Ecto Navegacao entre LiveViews via router	Concluido	lib/bigz/habits/habit.ex , lib/bigz/habits/checkin.ex
RT07	Phoenix	Concluido	lib/bigz_web/router.ex

Modulo A - Joao Pedro Mendes Moreira

Codigo	Requisito	Status	Arquivo principal
RF01	Cadastro com nome, e-mail e senha via mix phx.gen.auth	Concluido	lib/bigz/accounts/user.ex
RF02	Login e logout com sessao persistente Perfil com nome, bio editavel e	Concluido	lib/bigz_web/user_auth.ex
RF03	pontuacao total acumulada	Concluido	lib/bigz_web/live/user_live/profile.ex

Modulo B - Luiz Carlos

Codigo	Requisito	Status	Arquivo principal
RF04	Cadastro de habitos com nome, descricao, categoria e pontuacao	Concluido	lib/bigz/habits/habit.ex
RF05	Listagem de habitos com filtro por categoria Edicao e remocao de habitos	Concluido	lib/bigz_web/live/habit_live/index.ex
RF06	pelo proprio usuario	Concluido	lib/bigz/habits.ex

Modulo C - Ricardo Andre

Codigo	Requisito	Status	Arquivo principal
RF07	Check-in diario com impedimento de duplicidade no mesmo dia	Concluido	<code>lib/bigz/habits.ex</code> , <code>lib/bigz/habits/checkin.ex</code>
RF08	Dashboard pessoal com historico de check-ins e pontuacao semanal	Concluido	<code>lib/bigz_web/live/home_live/index.ex</code>
RF09	Feed da comunidade em tempo real via PubSub	Concluido	<code>lib/bigz_web/live/community_live/index.ex</code>

Notas Tecnicas de Implementacao

RF07 - Seguranca do check-in

O `user_id` e sempre extraido de `current_scope` (sessao autenticada) e nunca aceito como entrada do usuario. A data e definida exclusivamente no servidor via `Date.utc_today()`. A unicidade e garantida por indice composto no banco (`user_id + habit_id + checkin_date`) e pelo changeset Ecto, que retorna a mensagem "Voce ja registrou este habito hoje." em caso de duplicidade. A validacao no banco protege contra race conditions que a validacao em aplicacao nao conseguiria detectar.

RF08 - Pontuacao sem campo redundante

A pontuacao do usuario e calculada via `JOIN` entre `checkins` e `habits` com `SUM(h.points)`. O campo `score` da tabela `users` existe no schema mas propositalmente nunca e atualizado, evitando duas fontes de verdade no banco de dados. A agregacao por semana ISO e feita em Elixir com `Date.day_of_week/1`, sem depender de funcoes especificas do PostgreSQL como `date_trunc`.

RF09 - Atualizacao em tempo real

O `CommunityLive.Index` subscreve ao topico PubSub "`checkins:community`" somente quando o socket WebSocket esta conectado (`connected?(socket)`), evitando subscricoes durante o render HTTP inicial. O `handle_info/2` usa `stream_insert` com `at: 0` para adicionar o check-in ao topo do feed. O mecanismo de Phoenix Streams garante deduplicacao automatica por DOM id. O nome do usuario e exibido mas o e-mail nunca aparece na interface, preservando a privacidade.

Declaracao de Uso de Inteligencia Artificial

A equipe utilizou a ferramenta **Claude (Anthropic)** como assistente de aprendizado durante o desenvolvimento do projeto. O uso foi restrito a esclarecimento de duvidas conceituais, entendimento de documentacao e explicacoes sobre comportamento de funcoes do framework. Nenhum membro utilizou a ferramenta para gerar trechos de codigo diretamente.

As conversas abaixo representam as principais consultas realizadas por cada membro, demonstrando o uso da IA como recurso educacional.

Pergunta 1

"Quando rodo `mix phx.gen.auth`, o gerador cria uma tabela chamada `users_tokens`. Para que ela serve exatamente? Preciso de uma tabela separada so para isso?"

Resposta:

A tabela `users_tokens` armazena tres tipos diferentes de tokens: tokens de sessao (context: "session"), tokens de confirmacao de e-mail (context: "confirm") e tokens de magic link (context: "login"). Ter uma tabela separada permite revogar sessoes individualmente sem alterar o registro do usuario, alem de suportar multiplas sessoes simultaneas em dispositivos diferentes. Cada token tem um contexto que define seu proposito e regras de expiracao. Por exemplo, tokens de sessao sao validos por 60 dias, enquanto tokens de confirmacao expiram em 7 dias. Sem essa tabela, voce precisaria guardar informacoes de sessao dentro do proprio registro do usuario, o que complicaria a revogacao granular.

Pergunta 2

"O Phoenix v1.8 tem um conceito de `Scope` que e diferente do `@current_user` que eu conhecia em versoes anteriores. Como o `Scope` funciona e por que ele foi adotado?"

Resposta:

O `Scope` e um struct que encapsula o contexto de autorizacao completo, nao apenas o usuario. Em vez de passar o usuario diretamente para funcoes de contexto como `Habits.create_habit(user, attrs)`, voce passa `Habits.create_habit(scope, attrs)`. O `Scope` carrega o campo `.user` e pode carregar outros campos como organizacao ou papel, facilitando multi-tenancy no futuro sem alterar a assinatura das funcoes. No Phoenix v1.7 e anteriores, o padrao era usar `@current_user` diretamente no socket, o que funcionava mas tornava a adicao de contexto adicional mais trabalhosa. Para um projeto single-tenant, o `Scope` basicamente age como um wrapper com o usuario dentro, mas o codigo ja fica preparado para crescer.

Pergunta 3

"Por que a senha minima e 12 caracteres no gerador padrao do Phoenix? Posso reduzir para 8 sem prejuizo?"

Resposta:

O limite de 12 caracteres segue as recomendacoes do NIST (publicacao SP 800-63B), que priorizou comprimento sobre complexidade. Pesquisas mostraram que senhas longas sao mais resistentes a ataques de forca bruta do que senhas curtas com caracteres especiais obrigatorios. O Phoenix usa PBKDF2 com salt automatico, que e intencionalmente lento para dificultar ataques offline, mas uma senha maior e sempre a melhor defesa em camadas. Tecnicamente voce pode reduzir para 8 alterando o `validate_length` no changeset, mas isso enfraquece a seguranca. A recomendacao e manter 12 ou aumentar.

Conversas - Luiz Carlos (Modulo B)

Pergunta 1

"Estou tentando construir uma query Ecto que filtra habitos por categoria, mas so quando o parametro de categoria for fornecido. Se nao houver filtro, quero retornar todos. Qual a forma mais idiomatica de fazer isso?"

Resposta:

A abordagem mais idiomatica em Ecto e compor a query de forma incremental usando o pipe operator. Voce comeca com a query base e aplica condicoes com `where` so quando o valor nao e nil ou vazio. Por exemplo:

```
query = from(h in Habit, order_by: [asc: h.name])

query =
  if category && category != "" do
    from(h in query, where: h.category == ^category)
  else
    query
  end

Repo.all(query)
```

E importante tambem validar que o valor de `category` pertence a uma lista permitida antes de usar em query, para evitar retornos inesperados. Isso e diferente de injecao SQL (Ecto parametriza automaticamente), mas garante que so valores validos sejam aceitos como filtro.

Pergunta 2

"Onde devo verificar autorizacao num LiveView? No `handle_event`, no `apply_action` ou dentro do contexto em `habits.ex`?"

Resposta:

A pratica recomendada e verificar em multiplas camadas, cada uma com responsabilidade diferente. O contexto (`lib/bigz/habits.ex`) e a ultima linha de defesa: `update_habit/3` deve verificar se `habit.user_id == scope.user.id` antes de executar qualquer alteracao no banco. Isso garante que nenhuma chamada ao contexto, independente de onde venha, consiga burlar a autorizacao. No LiveView, o `apply_action/3` pode verificar a mesma condicao e exibir uma mensagem amigavel antes de renderizar o formulario. A UI pode esconder botoes de edicao com condicionais, mas isso e apenas conveniencia visual, nao seguranca. As tres camadas juntas formam uma defesa em profundidade: UI esconde, LiveView valida cedo, contexto e a garantia final.

Pergunta 3

"Quando o filtro de categoria muda na listagem de habitos, o stream fica com itens antigos misturados com novos. Como fazer o stream substituir completamente a lista?"

Resposta:

Quando voce quer substituir todos os itens do stream em vez de acrescentar, use a opcao `reset: true` na chamada ao stream. Por exemplo, no `handle_params/3` do LiveView:

```
def handle_params(params, _uri, socket) do
  category = params["category"]
  habits = Habits.list_habits(category)
  {:noreply, stream(socket, :habits, habits, reset: true)}
end
```

Com `reset: true`, o Phoenix substitui todos os itens existentes no stream pelos novos. Sem esse flag, o stream apenas acrescenta novos itens sem remover os anteriores, causando duplicatas ou itens desatualizados. A opcao `reset: true` foi adicionada justamente para cenarios de filtro como esse.

Conversas - Ricardo Andre (Modulo C)

Pergunta 1

"Quero que quando o Ecto detectar violacao do unique constraint no banco, ele exiba uma mensagem personalizada em portugues em vez da mensagem tecnica padrao. Como configuro isso no changeset?"

Resposta:

Voce configura isso passando a opcao `message` para a funcao `unique_constraint` no changeset. O ponto critico e que o nome do indice no changeset precisa coincidir exatamente com o nome do indice criado na migration. Por exemplo:

```
def changeset(checkin, attrs) do
  checkin
  |> cast(attrs, [])
  |> validate_required([:user_id, :habit_id, :checkin_date])
  |> unique_constraint(
    [:user_id, :habit_id, :checkin_date],
    name: :checkins_user_id_habit_id_checkin_date_index,
    message: "Voce ja registrou este habito hoje."
  )
end
```

Quando o `Repo.insert` falha por violacao do constraint, o Ecto captura a execucao do banco, identifica o indice pelo nome e converte em um erro de changeset com a mensagem que voce definiu. Esse erro

aparece normalmente em `changeset.errors` e pode ser exibido na interface como qualquer outro erro de validacao.

Pergunta 2

"Li que devo usar `connected?(socket)` antes de subscrever ao PubSub no LiveView. Por que isso e necessario? O que acontece se eu subscrever sem essa verificacao?"

Resposta:

O LiveView executa o `mount/3` duas vezes: uma vez durante o render HTTP inicial (para enviar HTML ao navegador antes do WebSocket abrir) e uma segunda vez quando o WebSocket e estabelecido. Se voce subscrever ao PubSub na primeira execucao, o processo que recebe as mensagens e o processo temporario do render HTTP, que termina quando a resposta e enviada. As mensagens chegam mas sao descartadas, e a subscricao some com o processo. Na segunda execucao, quando o WebSocket esta de pe, voce tem um processo de longa duracao que realmente pode receber e processar as mensagens. O `connected?(socket)` retorna `false` na primeira chamada e `true` na segunda, permitindo subscrever apenas quando ha uma conexao real e persistente.

Pergunta 3

"Quero agrupar check-ins por semana ISO (segunda a domingo) para mostrar um grafico das ultimas 6 semanas. Nao quero usar `date_trunc` porque e especifico do PostgreSQL. Como fazer esse agrupamento em Elixir puro?"

Resposta:

Em Elixir voce pode calcular o inicio da semana ISO de qualquer data com uma formula simples: subtraia o numero do dia da semana menos 1 do dia atual. O modulo `Date` do Elixir tem `Date.day_of_week/1` que retorna 1 para segunda-feira e 7 para domingo, seguindo o padrao ISO 8601. Entao para encontrar a segunda-feira da semana:

```
def start_of_week(date) do
  Date.add(date, -(Date.day_of_week(date) - 1))
end
```

Com isso voce usa `Enum.group_by` para agrupar as datas pelo inicio da semana calculado:

```
rows
|> Enum.group_by(fn {date, _pts} -> start_of_week(date) end)
|> Enum.map(fn {week_start, entries} ->
  total = entries |> Enum.map(&elem(&1, 1)) |> Enum.sum()
  %{week_start: week_start, points: total}
end)
```


A vantagem é que a lógica fica em Elixir, funciona com qualquer banco de dados e é mais fácil de testar em isolamento.
